

图 6.7: Linux 文件系统各核心数据结构之间的引用关系。左半是文件系统侧：`file_system_type` 描述一种文件系统，把它的每个实例（`super_block`）用 `s_instances` 串成链，`s_type` 指回类型；一个 `super_block` 通过 `s_inodes` 挂起该文件系统内所有内存 `inode`，块设备上的文件系统还有 `s_bdev` 指向 `block_device`。每个 `inode` 用 `i_sb` 指回所属超级块、`i_mapping` 指向 `address_space`（页缓存）。右半是目录与进程侧：`dentry` 是路径中的一个目录项，`d_inode` 指向 `inode`，`d_parent/d_subdirs/d_child` 把目录项连成树，`d_alias` 让同一 `inode` 的多个硬链接挂在一条链上；`path` 把挂载点 `vfsmount` 与 `dentry` 配对。`task_struct` 经 `fs` 指向 `fs_struct`（保存 `root`、`pwd` 两个 `path`），经 `files` 指向 `files_struct`，后者用 `fdt` 指向 `fdtable`，`fd[i]` 数组的每一项指向一个打开的 `file`；`file` 再用 `f_path`、`f_mapping`、`f_op` 分别指回 `path`、`address_space` 与操作表。本项目把这张图简化成表 6.10 里的那几样：没有 `dentry` 缓存，`file_operations` 与 `inode_operations` 合并为一张 `inode_ops`。

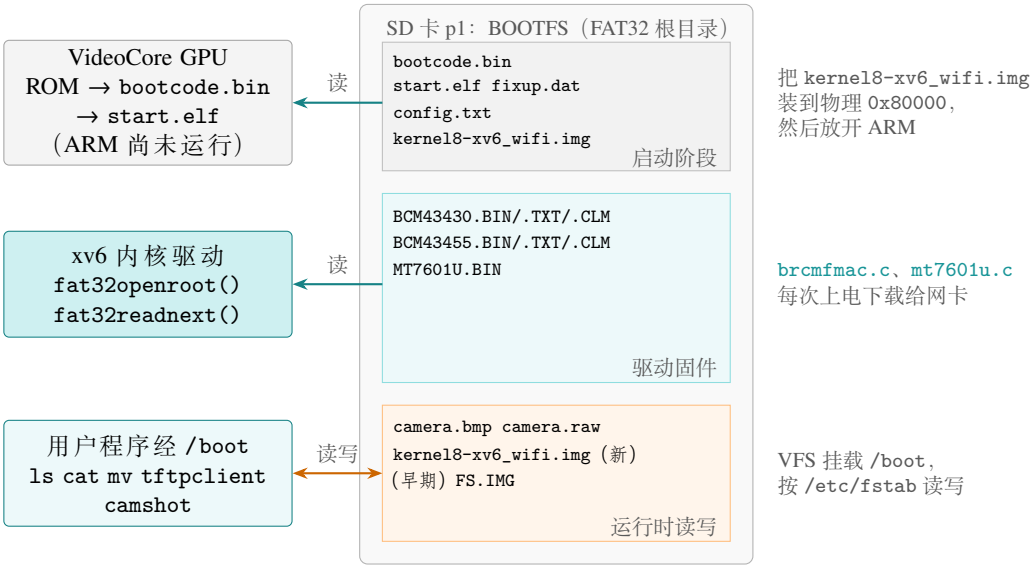


图 6.8: 启动分区里的文件和它们的三类读者。同一个 FAT32 分区，先被 GPU 固件读（ARM 还没开始执行），再被 xv6 内核驱动读，最后作为 `/boot` 被用户程序读写